CIN0145 – Programação Concorrente

Introdução

André Santos

CIn-UFPE

©André Santos

O que é concorrência?

- Um processo é um programa sequencial que executa uma sequência de comandos
- Um programa concorrente possui dois ou mais ***processos*** que cooperam na execução de uma tarefa

©André Santos

Processos cooperam através de comunicação

- usando variáveis compartilhadas
  - um processo escreve em uma variável e outro processo lê
- usando troca de mensagens
  - Um processo envia mensagens que outro processo recebe

©André Santos

Programas concorrentes

- Programas concorrentes são mais complexos que os programas sequenciais: difíceis de programar corretamente devido à interação entre processos

©André Santos

Como desenvolver programas concorrentes

- Através do uso de técnicas formais para especificar, programar e verificar programas
- Através do uso de construções de linguagens para expressar programas concorrentes: **semáforos, monitores, troca de mensagens**
- Através do conhecimento de classes de problemas clássicos envolvendo concorrência

©André Santos

Origem Histórica

- Evolução do hardware e consequente evolução na complexidade dos problemas
- Programação concorrente ad-hoc evoluindo para princípios e técnicas de programação concorrente

©André Santos

Sistemas Operacionais

- Surgimento de controladoras de I/O independentes (1960s) tornou natural a organização de sistemas operacionais como um programa concorrente
- Processos gerenciam dispositivos e a execução de tarefas dos usuários
- Processos implementados em máquinas com um único processador através de *multiprogramação*

©André Santos

Multiprogramação

- Processos executam um de cada vez, de forma alternada (*interleaved*):
  lê, processa, escreve
- Primeiros usos tinham o objetivo de alternar I/O e computação de vários programas dos usuários em um único computador
- inicialmente o chaveamento era ativado pela espera por I/O

©André Santos

Multiprogramação

- Atualmente o chaveamento é baseado em time-slicing
  - ⇒ compartilha o processador entre várias computações
  - ⇒ uso de timers/interrupções
  - ⇒*Preemptive Multitasking*
- Escalonador do Sistema Operacional
  - ⇒ determina ordem e prioridade de execução

©André Santos

Arquiteturas

- Com memória compartilhada – Multiprocessadores (*Multicore*)
  - *UMA – Uniform Memory Access*
  - *NUMA – Non-Uniform Memory Access*
- Com memória distribuída - Multicomputadores
  - *Nós* (processadores) conectados através de um hardware de chaveamento de mensagens de alta velocidade
- Redes de computadores
  - *Nós* com um ou mais processadores compartilhando uma rede de comunicação

©André Santos

Outros exemplos de aplicações concorrentes

- Sempre que a aplicação exige paralelismo real ou aparente
  - Sistemas de janelas
  - Sistemas de processamento de transações em bancos de dados multiusuário
  - Servidores de arquivos em uma rede
  - computação científica, com manipulação de grandes massas de dados
  - Sistemas de tempo real e sistemas críticos

©André Santos

Real-time Systems

- Embedded systems: aviões, radares, raios X, usinas
- processos com limitações de tempo de resposta (e.g. milisegundos)

©André Santos

Definições

- Programação paralela
  - Programas concorrentes, em multiprocessadores
  - Memória compartilhada (geralmente)
- Programação distribuída
  - Programação concorrente em que os processos se comunicam através de troca de mensagens ou chamada remota de procedimentos (RPC)
  - Memória distribuída

©André Santos

Programação Paralela – Pequena Granularidade

- Small-grained Concurrency
  - Execução de múltiplas instruções de máquina ao mesmo tempo em um processador
  - necessita de suporte de hardware para ser eficiente
    `E := (A*B) + (C+D);`
  - específica para uma arquitetura ou problema
  - **abordada em outros cursos da área de arquitetura de computadores**

©André Santos

Programação Paralela – Alta Granularidade

- Large-grained concurrency
  - Processos maiores, que individualmente fazem mais tarefas
  - *Cooperação* de vários processos, para resolução de problemas
  - Exemplo: shell do Unix

```
sort | uniq | format | print
```

©André Santos

Formas de Sincronização

- A comunicação entre processos cria a necessidade de sincronização
- Existem duas formas de sincronização:
  - **Exclusão mútua**
  - **Sincronização de condição**

©André Santos

Exclusão mútua

- Garantir que *regiões críticas* do programa, que acessam objetos compartilhados, não são executadas ao mesmo tempo.

©André Santos

Sincronização de Condição

- Garantir que um processo espera (se necessário) até que uma condição seja verdadeira.

©André Santos

Exemplo: Problema do Produtor-Consumidor

- Exige os dois tipos de sincronização
- Leitura/escrita no buffer não pode ocorrer ao mesmo tempo
  - Impede leitura de dados incompletos
- Espera caso o buffer esteja vazio ou cheio
  - Evita que os mesmos dados sejam lidos mais de uma vez ou sobrescritos

©André Santos

Estado de um programa concorrente

- Consiste nos Valores das variáveis do programa (explícitas e implícitas)
- Execução de um programa transforma o estado
- cada comando do programa implica na execução de uma ou mais ***ações atômicas***, que fazem transformações (indivisíveis) de estado.

©André Santos

Execução de um programa concorrente

- Gera uma sequência de ações atômicas, que é um *interleaving* das sequências de ações atômicas de cada processo componente
- Cada execução de um programa concorrente gera uma *história* (*trace*) de execução diferente
- **Mesmo programas paralelos podem ser modelados desta forma**

©André Santos

Execução de um programa concorrente

- Enorme número de histórias (traces) possíveis
  - 3 procedimentos, 2 instruções cada: 90 histórias
  - 3 procedimentos, 10 instruções cada: 1000 possíveis estados do programa
- *n* processos, com *m* instruções atômicas cada: $(n*m)!/(m!)^n$

©André Santos

Interleave arbitrário de instruções

- Programa concorrente deve estar **correto para todo *interleaving***
- *debugging* é muito mais complicado
- É preciso usar técnicas de programação e verificação que garantam que um programa estará correto sob qualquer *interleaving*.

©André Santos

Corretude

- Programa incorreto: basta apresentar uma sequência que leve ao erro
- Prova de corretude: precisa provar que o programa está correto para qualquer sequência escolhida pelo escalonador

©André Santos

Objetivo da Sincronização

- Restringir as possíveis histórias de um programa concorrente para aquelas que se deseja obter

©André Santos

Objetivo da Sincronização

- Exclusão mútua combina ações atômicas em regiões críticas que parecem ser atômicas, evitando o *interleave* com a execução de outras regiões críticas.
- Sincronização de condição atrasa um processo até um estado em que ele possa continuar executando

©André Santos

Objetivo da Sincronização

- As duas formas de sincronização causam atrasos na execução de processos e consequentemente restringem o conjunto de ações atômicas que são elegíveis de serem executadas

©André Santos

Propriedades de programas

- Atributo que é verdade para toda possível história do programa, e portanto para toda execução de um programa
- Toda propriedade pode ser formulada em termos de dois tipos especiais de propriedades: ***safety* e *liveness***

©André Santos

Safety properties

- “programa nunca entra em um estado indesejado”
- i.e. em que variáveis tem valores indesejados
- Devem ser sempre verdade

©André Santos

Liveness Properties

- “o programa em algum momento entrará no estado desejado”
- i.e. um estado em que todas as variáveis tem valores desejados

©André Santos

Propriedades de programas

- Corretude Parcial – *safety property*
  - Se um programa termina, o estado final é correto, i.e. o resultado correto foi computado
- Terminação – *liveness property*
  - Um dado programa terminará, i.e. toda história do programa é finita

©André Santos

Propriedades de programas

- Corretude Total
  - Um dado programa sempre terminará com uma resposta correta
- Exclusão mútua – *safety property*
  - No máximo um processo está executando sua região crítica
  - algumas sequências de instruções não podem ser interleaved
  - uma sequência deve terminar antes que a outra comece
  - Exemplo: acesso a impressoras, dados, serviços

©André Santos

Propriedades de programas

sejam P(x), Q(x,y) propriedades sobre as variáveis de entrada e saída

- Corretude Parcial

$(P(a) \wedge terminates(Prog(a,b))) \supset Q(a,b)$

- Corretude Total

$P(a) \supset (terminates(Prog(a,b)) \wedge Q(a,b))$

- Terminação

©André Santos

Exemplo

SQR(x,y) (a raiz quadrada de x é y)

- Corretude Parcial

$(a \geq 0 \wedge \text{terminates}(SQR(a,b))) \supset b = \sqrt{a}$

- Corretude Total

$a \geq 0 \supset (\text{terminates}(SQR(a,b)) \wedge b = \sqrt{a})$

©André Santos

Propriedades de programas

- Ausência de *Deadlock* – Safety property
  - os processos não podem ficar todos bloqueados ao mesmo tempo

©André Santos

Propriedades de programas

- Ausência de *(individual) starvation – liveness property*
  - se um processo requisita um recurso, em algum momento ele vai conseguir acessa-lo

©André Santos

Propriedades de programas

- *Fairness - liveness property*
- *weak fairness*
  - se um processo faz uma solicitação contínua, em algum momento a receberá
- *strong fairness*
  - se um processo faz uma solicitação infinitamente frequente, em algum momento a receberá

©André Santos

Fairness

- *linear waiting*
  - *se um processo faz uma solicitação, ele a receberá antes que qualquer outro processo a receba mais de uma vez*
- *FIFO*
  - *se um processo faz uma solicitação, ele a receberá antes que qualquer processo que faça uma solicitação posterior*

©André Santos

Como provar que um programa satisfaz uma propriedade

- Testes?
  - Revelam erros, não revelam ausência de erros
- *Operational Reasoning*
  - Análise exaustiva de casos
- *Assertional Reasoning*
  - Fórmulas de lógica de predicados chamadas asserções são usadas para caracterizar conjuntos de estados.
  - Ações transformam predicados

©André Santos